



TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



Disjoint sets

Nguyễn Khánh Phương

Computer Science department
School of Information and Communication technology
E-mail: phuongnk@soict.hust.edu.vn

Course outline

1. Duplicate Customer Account Consolidation
2. Longest consecutive subsequence
3. Minimum spanning tree - Kruskal algorithm
4. Islands
5. Friends

2

1. Duplicate customer account consolidation

An e-commerce platform stores N customer account IDs, numbered from 1 to N . Initially, every account is assumed to belong to a different customer. However, due to multiple registrations, one real customer may own several accounts. The platform periodically verifies that two accounts belong to the same person and must merge their customer profiles. When two profiles are merged, **all accounts already associated with either profile become part of one common profile**.

You must process Q operations of the following types:

- **MERGE x y** : The system confirms that accounts x and y belong to the same customer. Merge their profiles. If two accounts are already in the same profile, a MERGE operation does not change anything.
- **SAME x y** : Determine whether accounts x and y currently belong to the same customer profile.
- **SIZE x** : Output the number of accounts currently contained in the customer profile of account x .
- **COUNT**: Output the current number of distinct customer profiles in the system.

3

1. Duplicate customer account consolidation

Input

- The first line contains two integers: N Q where: $1 \leq N, Q \leq 200\,000$
- The next Q lines describe the operations in one of the following formats:

MERGE x y

SAME x y

SIZE x

COUNT

For all operations involving account IDs: $1 \leq x, y \leq N$

Output

For each operation:

- **SAME x y** : print YES if x and y belong to the same profile; otherwise print NO.
- **SIZE x** : print the number of accounts in the profile containing x .
- **COUNT**: print the number of distinct customer profiles currently stored in the system.

4

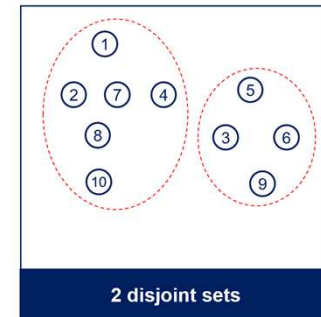
1. Duplicate customer account consolidation

Input	Output	Explanation
8 11	8	Initially, all accounts are separate:
COUNT	NO	$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\}$
SAME 1 2	3	Thus, COUNT will output 8, SAME 1 2 will output NO
MERGE 1 2	YES	After:
MERGE 2 4	NO	MERGE 1 2
SIZE 1	5	MERGE 2 4
SAME 1 4	4	accounts 1, 2, and 4 belong to the same profile: $\{1, 2, 4\}$
SAME 1 5		Therefore: SIZE 1 outputs 3. Notice that SIZE 2 and SIZE 4 would also output 3.
MERGE 5 6		Later, after:
MERGE 4 6		MERGE 5 6
SIZE 5		MERGE 4 6
COUNT		the profiles containing $\{1, 2, 4\}$ and $\{5, 6\}$ are consolidated: $\{1, 2, 4, 5, 6\}$
		Thus, SIZE 5 outputs 5. At the end, the remaining profiles are: $\{1, 2, 4, 5, 6\}, \{3\}, \{7\}, \{8\}$
		so COUNT outputs 4.

5

Disjoint set data structure

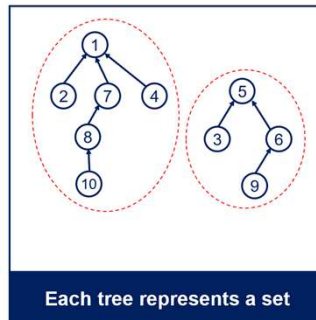
- Disjoint-Set: a data structure representing sets that do not intersect with two main operations
 - find(x)**: returns the identifier of the set containing x
 - union(r1, r2)**: merge two sets of identifiers r1 and r2 into one set



6

Disjoint set data structure

- Disjoint-Set: a data structure representing sets that do not intersect with two main operations
 - find(x)**: returns the identifier of the set containing x
 - union(r1, r2)**: merge two sets of identifiers r1 and r2 into one set
- Each set is represented by a rooted tree
 - Each node of the tree is an element
 - Each node x has a unique parent node p[x] (the parent of the root node is itself)
 - The root node is the identifier of the set

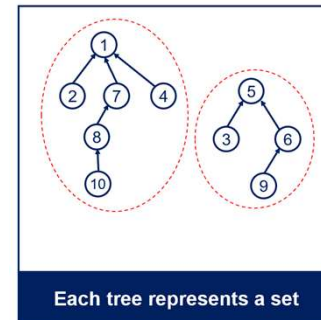


x	1	2	3	4	5	6	7	8	9	10
parent[x]	1	1	5	1	5	5	1	7	6	8

7

Disjoint set data structure

- find(x)**: returns the identifier of the set containing x
 - Starting from x, continuously access the parent node to find the root node
 - Complexity is proportional to the length of the path from x to the origin



```

find(x){
    while (x != parent[x])
        x = parent[x];
    return x;
}

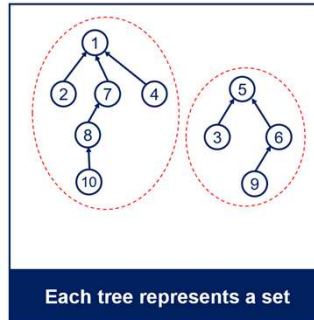
```

8

Disjoint set data structure

find(x): returns the identifier of the set containing x

- Starting from x, continuously access the parent node to find the root node
- Complexity is proportional to the length of the path from x to the origin
- Path Compression**: in the process of going from x to the root, nodes on the path will be attached as direct children of the root node → reduce the path length from these nodes to the root in the following iterations.



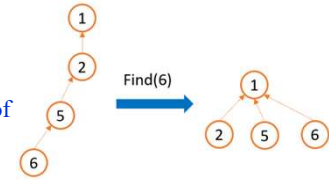
```
find(x){
  while (x != parent[x])
    x = parent[x];
  return x;
}
```

```
find(x){
  if(x != parent[x])
    parent[x] = find(parent[x]);
  return parent[x];
}
```

Disjoint set data structure

find(x): returns the identifier of the set containing x

- Starting from x, continuously access the parent node to find the root node
- Complexity is proportional to the length of the path from x to the origin
- Path Compression**: in the process of going from x to the root, nodes on the path will be attached as direct children of the root node → reduce the path length from these nodes to the root in the following iterations.



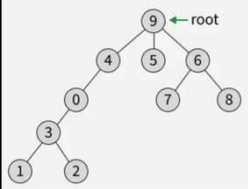
```
find(x){
  while (x != parent[x])
    x = parent[x];
  return x;
}
```

```
find(x){
  if (x != parent[x])
    parent[x] = find(parent[x]);
  return parent[x];
}
```

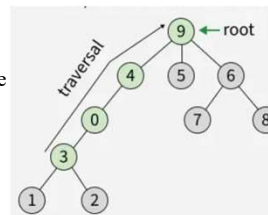
Disjoint set: Path compression

```
find(x) {
  if (x != parent[x])
    parent[x] = find(parent[x]) #path compression
  return parent[x];
}
```

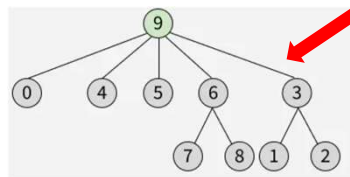
Set {0, 1...9} is represented as



Call find(3): traverse up and find 9 is id of the tree



- The idea of path compression is to make the found root as parent of x so that we don't have to traverse all intermediate nodes again. If x is root of a subtree, then path (to root) from all nodes under x also compresses. → It speeds up the data structure by compressing the height of the trees.

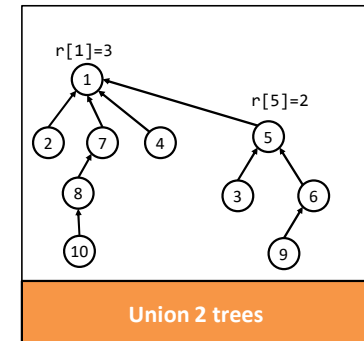


When find(3) is terminated, path compression is already applied → we get directly link element 0 and 3 to 9

Disjoint set data structure

To reduce the height of tree:

- When merging two trees, we join the tree with the lower height to the tree with the higher height.
- To record the height of a node x a tree, use the additional variable $r[x]$



Disjoint set

```

1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2.   if (x != y) then
3.     parent[x] = y; //make id of x's set be the id of y's set

```



```

1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2.   if (x != y) then
3.     if (r[x] > r[y]) then parent[y] = x;
4.     else{
5.       parent[x] = y;
6.       if (r[x] == r[y]) then r[y] = r[y] + 1; Union by rank to reduce
7.     }                                     the height of tree
8.}

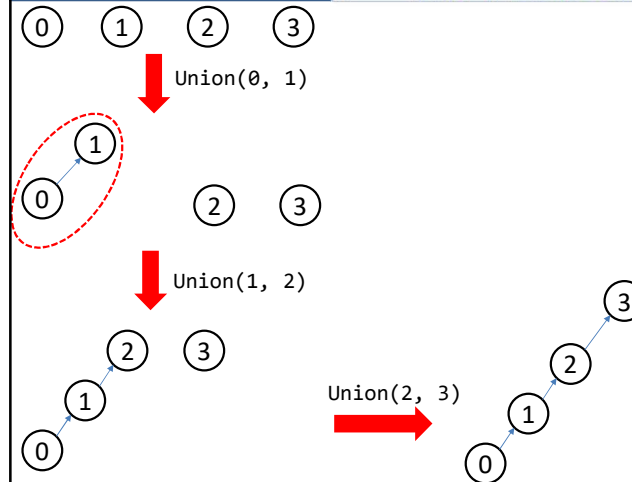
```

Disjoint set

```

1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2.   if (x != y) then
3.     parent[x] = y; //make id of x's set be the id of y's set

```

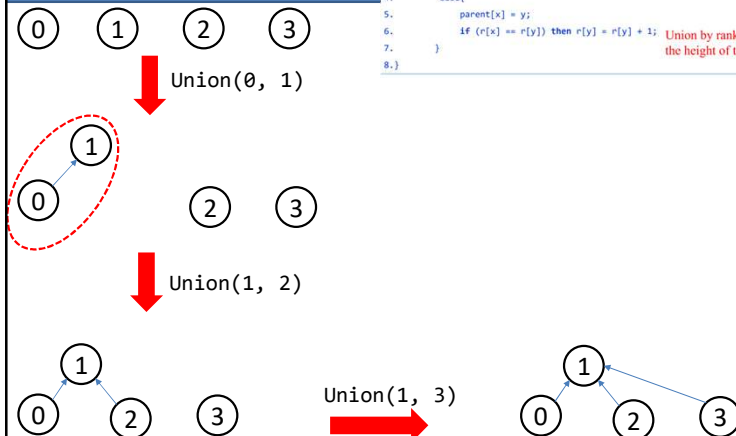


Disjoint set

```

1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2.   if (x != y) then
3.     if (r[x] > r[y]) then parent[y] = x;
4.     else{
5.       parent[x] = y;
6.       if (r[x] == r[y]) then r[y] = r[y] + 1; Union by rank to reduce
7.     }                                     the height of tree
8.}

```



Disjoint set data structure

Disjoint Set: using 2 arrays:

- $parent[x]$: parent of node x on the tree containing x (if x is the root then $parent[x] = x$)
- $r[x]$: height of node x in the tree containing x

```

1. makeSet(x) { //Create a set containing x
2.   parent[x] = x; r[x] = 0;
3. }
4. find(x) { //return the identifier of the set containing x
5.   if (x != parent[x])
6.     parent[x] = find(parent[x]);
7.   return parent[x];
8. }
9. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
10.   if (x != y) then
11.     if (r[x] < r[y]) then parent[x] = y;
12.     else{
13.       parent[y] = x;
14.       if (r[x] == r[y]) then r[x] = r[x] + 1;
15.     }

```

Disjoint set data structure

Disjoint Set: using 2 arrays:

- **parent[x]**: parent of node x on the tree containing x (if x is the root then parent[x] = x)
- **sz[x]**: number of nodes in the tree containing x

```

1. makeSet(x) { //Create a set containing x
2.     parent[x] = x; sz[x] = 0;
3. }
4. find(x) { //return the identifier of the set containing x
5.     if (x != parent[x]) parent[x] = find(parent[x]);
6.     return parent[x];
7. }
8. union(x, y) { //union 2 sets with identifiers x and y
9.     if (x != y)
10.        if (sz[x] < sz[y]) {sz[y] += sz[x]; parent[x] = y;}
11.        else {
12.            sz[x] += sz[y];
13.            parent[y] = x;
14.        }
15. }

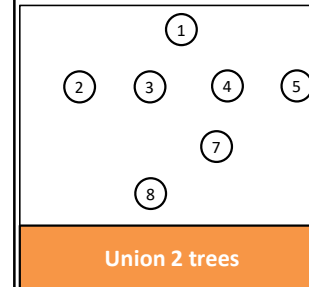
```

1. Duplicate customer account consolidation

Example: {1}, {2}, {3}, {4}, {5}, {6}, {7}, {8}

SAME 1 2

same_profile(1, 2)



x	1	2	3	4	5	6	7	8
parent[x]	1	2	3	4	5	6	7	8

```

find(x){
    if (x != parent[x])
        parent[x] = find(parent[x]);
    return parent[x];
}

// Check whether x and y belong to the same customer profile
bool same_profile(int x, int y) {
    return find(x) == find(y);
}

```

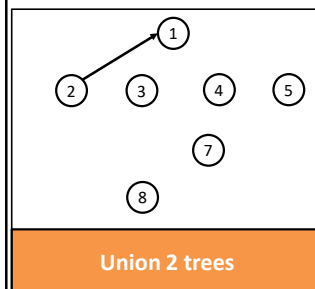
18

1. Duplicate customer account consolidation

Example: {1}, {2}, {3}, {4}, {5}, {6}, {7}, {8}

MERGE 1 2

merge_account(1, 2)



```

union(x, y) { //union 2 sets with identifiers x and y
    if (x != y)
        if (sz[x] < sz[y]) {sz[y] += sz[x]; parent[x] = y;}
        else {
            sz[x] += sz[y];
            parent[y] = x;
        }
    }
}

```

```

bool merge_account(int a, int b) {
    int root_a = find(a);
    int root_b = find(b);
    //They already belong to the same customer profile
    if (root_a == root_b) return false;

    union(root_a, root_b);
    number_of_profiles--;
    return true;
}

```

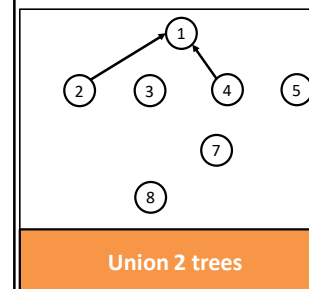
19

1. Duplicate customer account consolidation

Example: {1, 2}, {3}, {4}, {5}, {6}, {7}, {8}

MERGE 2 4

merge_account(2, 4)



```

union(x, y) { //union 2 sets with identifiers x and y
    if (x != y)
        if (sz[x] < sz[y]) {sz[y] += sz[x]; parent[x] = y;}
        else {
            sz[x] += sz[y];
            parent[y] = x;
        }
    }
}

```

```

bool merge_account(int a, int b) {
    int root_a = find(a);
    int root_b = find(b);
    //They already belong to the same customer profile
    if (root_a == root_b) return false;

    union(root_a, root_b);
    number_of_profiles--;
    return true;
}

```

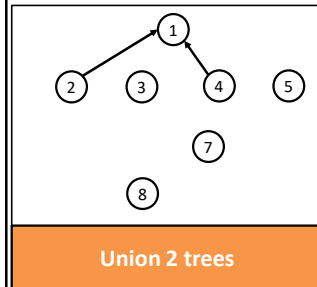
20

1. Duplicate customer account consolidation

Example: {1, 2, 4}, {3}, {5}, {6}, {7}, {8}

SIZE 4

account_profile_size(4)



```
int account_profile_size(int a) {
    return sz[find(a)];
}
```

```
find(x){
    if (x != parent[x])
        parent[x] = find(parent[x]);
    return parent[x];
}
```

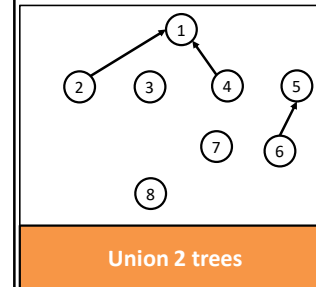
21

1. Duplicate customer account consolidation

Example: {1, 2, 4}, {3}, {5}, {6}, {7}, {8}

MERGE 5 6

merge_account(5, 6)



```
union(x, y) { //union 2 sets with identifiers x and y
    if (x != y)
        if (sz[x] < sz[y]) {sz[y] += sz[x]; parent[x] = y;}
        else {
            sz[x] += sz[y];
            parent[y] = x;
        }
}
```

```
bool merge_account(int a, int b) {
    int root_a = find(a);
    int root_b = find(b);
    //They already belong to the same customer profile
    if (root_a == root_b) return false;

    union(root_a, root_b);
    number_of_profiles--;
    return true;
}
```

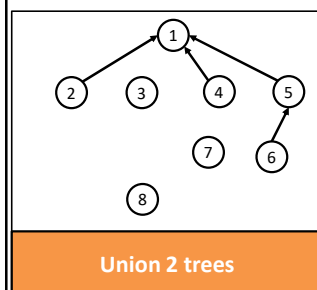
22

1. Duplicate customer account consolidation

Example: {1, 2, 4}, {3}, {5, 6}, {7}, {8}

MERGE 4 6

merge_account(4, 6)



```
union(x, y) { //union 2 sets with identifiers x and y
    if (x != y)
        if (sz[x] < sz[y]) {sz[y] += sz[x]; parent[x] = y;}
        else {
            sz[x] += sz[y];
            parent[y] = x;
        }
}
```

```
bool merge_account(int a, int b) {
    int root_a = find(a);
    int root_b = find(b);
    //They already belong to the same customer profile
    if (root_a == root_b) return false;

    union(root_a, root_b);
    number_of_profiles--;
    return true;
}
```

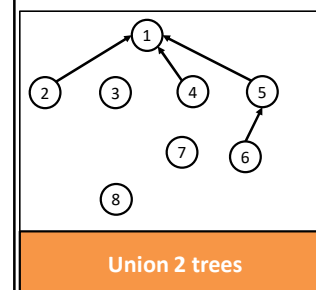
23

1. Duplicate customer account consolidation

Example: {1, 2, 4, 5, 6}, {3}, {7}, {8}

MERGE 7 6

merge_account(7, 6)
find(7); find(6)



```
union(x, y) { //union 2 sets with identifiers x and y
    if (x != y)
        if (sz[x] < sz[y]) {sz[y] += sz[x]; parent[x] = y;}
        else {
            sz[x] += sz[y];
            parent[y] = x;
        }
}

find(x) { //path
    if (x != parent[x])
        parent[x] = find(parent[x]);
    return parent[x];
}

bool merge_account(int a, int b) {
    int root_a = find(a);
    int root_b = find(b);
    //They already belong to the same customer profile
    if (root_a == root_b) return false;

    union(root_a, root_b);
    number_of_profiles--;
    return true;
}
```

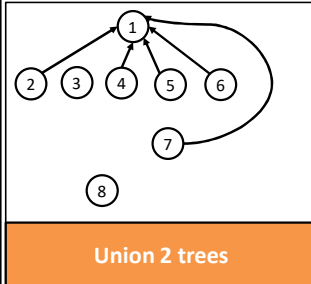
24

1. Duplicate customer account consolidation

Example: {1, 2, 4, 5, 6}, {3}, {7}, {8}

MERGE 7 6

```
merge_account(7, 6)
  find(7); find(6)
  union(7, 6)
```



```
union(x, y) { //union 2 sets with identifiers x and y
    if (sz[x] < sz[y]) {
        sz[y] += sz[x]; parent[x] = y;
    }
    else {
        sz[x] += sz[y]; parent[y] = x;
    }
}

find(x) { //path
    if (x != parent[x])
        parent[x] = find(parent[x]);
    return parent[x];
}

bool merge_account(int a, int b) {
    int root_a = find(a);
    int root_b = find(b);
    //They already belong to the same customer profile
    if (root_a == root_b) return false;

    union(root_a, root_b);
    number_of_profiles--;
    return true;
}
```

25

Summary: Disjoint set

- Path compression: to improve computation time of function find(x)

```
1. find(x) { //return id of the set containing x
2.   while (x != parent[x]) do x = parent[x];
3.   return x;
4. }
```

↓ Path compression

```
find(x) {
    if (x != parent[x])
        parent[x] = find(parent[x]) #path compression
    return parent[x];
}
```

Summary: Disjoint set

1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2. if (x != y) then
3. parent[x] = y; //make id of x's set be the id of y's set



1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2. if (x != y) then
3. if (r[x] > r[y]) then parent[y] = x;
4. else{
5. parent[x] = y;
6. if (r[x] == r[y]) then r[y] = r[y] + 1; Union by rank to reduce the height of tree
7. }
8. }

Disjoint set data structure

CASE 1: If use union(x, y) not by rank/size and find(x) not compression:

- find(x): could take $O(|N|)$ in a worst-case scenario (a long, skinny tree).
- union(x, y): Each union(x, y) operation takes $O(1)$ because we just update the parent.

```
1. makeSet(x) { //Create a set containing x
2.   parent[x] = x; r[x] = 0;
3. }
4. find(x) { //return id of the set containing x; not compression
5.   while (x != parent[x]) do x = parent[x];
6.   return x;
7. }
8. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y; not by rank
9.   if (x != y) then
10.     parent[x] = y; //make id of x's set be the id of y's set
```

Disjoint set data structure

CASE 2: If use union(x, y) by rank/size and find(x) not compression:

- find(x): $O(\log|N|)$
- union(x, y): $O(1)$ to update parent and rank.

```
1. makeSet(x) { //Create a set containing x
2.     parent[x] = x; r[x] = 0;
3. }
4. find(x) { //return id of the set containing x; not compression
5.     while (x != parent[x]) do x = parent[x];
6.     return x;
7. }
8. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y; union by rank
9.     if (x != y) then
10.         if (r[x] > r[y]) then parent[y] = x;
11.         else{
12.             parent[x] = y;
13.             if (r[x] == r[y]) then r[y] = r[y] + 1;
14.         }
15. }
```

Disjoint set data structure

CASE 3: If use union(x, y) by rank/size and find(x) path compression:

- find(x): $\alpha(|N|)$ where $\alpha(|N|)$ is the **inverse Ackermann function** (If $|N| \approx 10^{80}$ then $\alpha(|N|) \leq 5$), so find(x) takes almost constant $O(1)$.
- union(x, y): $O(1)$

```
1. makeSet(x) { //Create a set containing x
2.     parent[x] = x; r[x] = 0;
3. }
4. find(x) {
5.     if (x != parent[x])
6.         parent[x] = find(parent[x]) //path compression
7.     return parent[x];
8. }
9. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y; union by rank
10.     if (x != y) then
11.         if (r[x] > r[y]) then parent[y] = x;
12.         else{
13.             parent[x] = y;
14.             if (r[x] == r[y]) then r[y] = r[y] + 1;
15.         }
16. }
```

Course outline

1. Duplicate Customer Account Consolidation
2. Longest consecutive subsequence
3. Minimum spanning tree - Kruskal algorithm
4. Islands
5. Friends

2. Longest consecutive subsequence

The problem: Given an unsorted array of nonnegative integers, find the length of the longest consecutive elements sequence.

Example 1:

Input: nums = [100, 4, 200, 1, 3, 2, 1]

Output: 4

Explanation: The longest consecutive elements sequence is [1, 2, 3, 4].

Therefore its length is 4.

Example 2:

Input: nums = [0, 3, 7, 2, 5, 8, 4, 6, 0, 1, 100]

Output: 9

2. Longest consecutive subsequence

The problem: Given an unsorted array of nonnegative integers, find the length of the longest consecutive elements sequence.

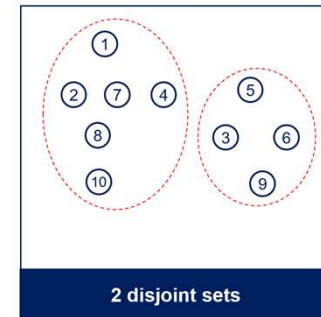
Method 1: sort the array, then scan from left to right to find consecutive sequences. The one with the longest length will be returned.

Method 2: use disjoint sets data structure

33

Disjoint set data structure

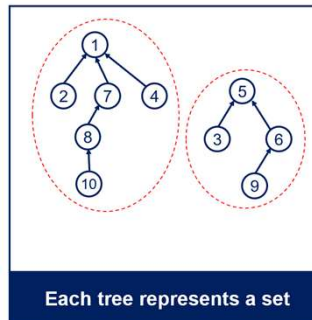
- Disjoint-Set: a data structure representing sets that do not intersect with two main operations
 - find(x)**: returns the identifier of the set containing x
 - union(r1, r2)**: merge two sets of identifiers r1 and r2 into one set



34

Disjoint set data structure

- Disjoint-Set: a data structure representing sets that do not intersect with two main operations
 - find(x)**: returns the identifier of the set containing x
 - union(r1, r2)**: merge two sets of identifiers r1 and r2 into one set
- Each set is represented by a rooted tree
 - Each node of the tree is an element
 - Each node x has a unique parent node p[x] (the parent of the root node is itself)
 - The root node is the identifier of the set

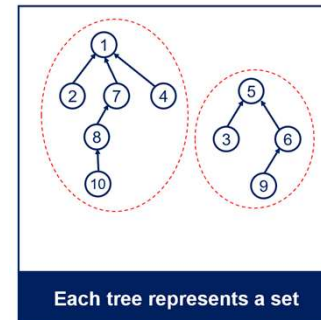


x	1	2	3	4	5	6	7	8	9	10
parent[x]	1	1	5	1	5	5	1	7	6	8

35

Disjoint set data structure

- find(x)**: returns the identifier of the set containing x
 - Starting from x, continuously access the parent node to find the root node
 - Complexity is proportional to the length of the path from x to the origin



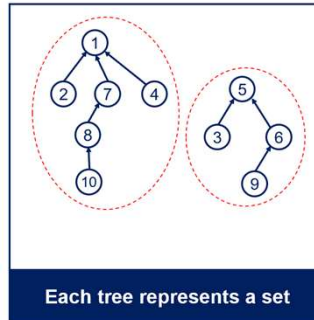
```
find(x){
    while (x != parent[x])
        x = parent[x];
    return x;
}
```

36

Disjoint set data structure

find(x): returns the identifier of the set containing x

- Starting from x, continuously access the parent node to find the root node
- Complexity is proportional to the length of the path from x to the origin
- Path Compression**: in the process of going from x to the root, nodes on the path will be attached as direct children of the root node → reduce the path length from these nodes to the root in the following iterations.



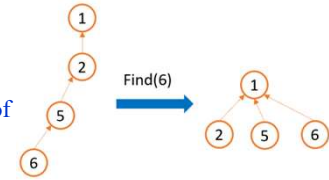
```
find(x){
  while (x != parent[x])
    x = parent[x];
  return x;
}
```

```
find(x){
  if(x != parent[x])
    parent[x] = find(parent[x]);
  return parent[x];
}
```

Disjoint set data structure

find(x): returns the identifier of the set containing x

- Starting from x, continuously access the parent node to find the root node
- Complexity is proportional to the length of the path from x to the origin
- Path Compression**: in the process of going from x to the root, nodes on the path will be attached as direct children of the root node → reduce the path length from these nodes to the root in the following iterations.



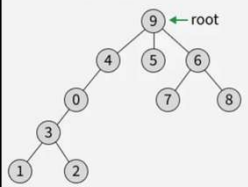
```
find(x){
  while (x != parent[x])
    x = parent[x];
  return x;
}
```

```
find(x){
  if(x != parent[x])
    parent[x] = find(parent[x]);
  return parent[x];
}
```

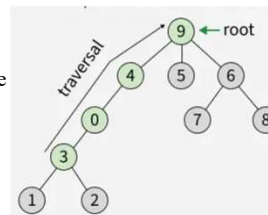
Disjoint set: Path compression

```
find(x) {
  if (x != parent[x])
    parent[x] = find(parent[x]) #path compression
  return parent[x];
}
```

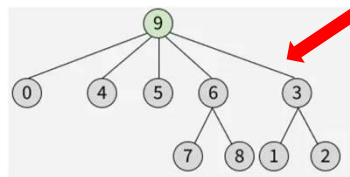
Set {0, 1...9} is represented as



Call find(3): traverse up and find 9 is id of the tree



- The idea of path compression is to make the found root as parent of x so that we don't have to traverse all intermediate nodes again. If x is root of a subtree, then path (to root) from all nodes under x also compresses. → It speeds up the data structure by compressing the height of the trees.

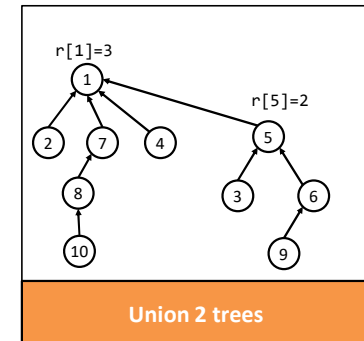


When find(3) is terminated, path compression is already applied → we get directly link element 0 and 3 to 9

Disjoint set data structure

To reduce the height of tree:

- When merging two trees, we join the tree with the lower height to the tree with the higher height.
- To record the height of a node x a tree, use the additional variable r[x]



Disjoint set

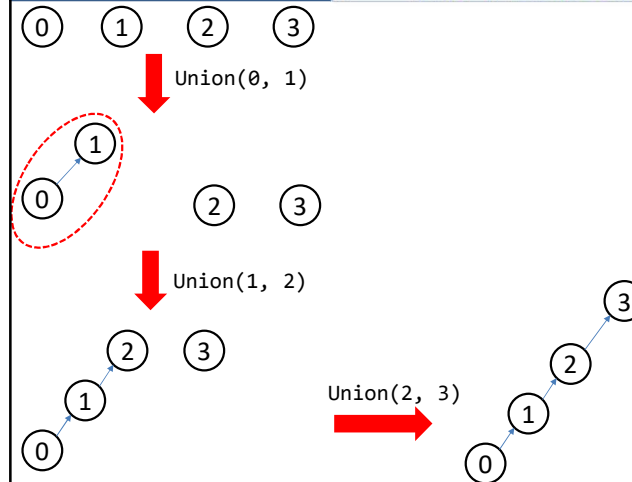
```
1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2.   if (x != y) then
3.     parent[x] = y; //make id of x's set be the id of y's set
```



```
1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2.   if (x != y) then
3.     if (r[x] > r[y]) then parent[y] = x;
4.     else{
5.       parent[x] = y;
6.       if (r[x] == r[y]) then r[y] = r[y] + 1; Union by rank to reduce
7.     }                                     the height of tree
8.}
```

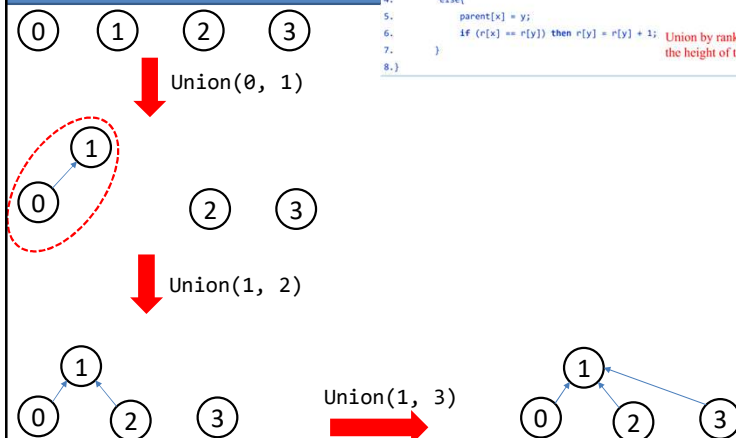
Disjoint set

```
1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2.   if (x != y) then
3.     parent[x] = y; //make id of x's set be the id of y's set
```



Disjoint set

```
1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2.   if (x != y) then
3.     if (r[x] > r[y]) then parent[y] = x;
4.     else{
5.       parent[x] = y;
6.       if (r[x] == r[y]) then r[y] = r[y] + 1; Union by rank to reduce
7.     }                                     the height of tree
8.}
```



Disjoint set data structure

Disjoint Set: using 2 arrays:

- parent[x]: parent of node x on the tree containing x (if x is the root then parent[x] = x)
- r[x]: height of node x in the tree containing x

```
1. makeSet(x) { //Create a set containing x
2.   parent[x] = x; r[x] = 0;
3. }
4. find(x) { //return the identifier of the set containing x
5.   if (x != parent[x])
6.     parent[x] = find(parent[x]);
7.   return parent[x];
8. }
9. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
10.   if (x != y) then
11.     if (r[x] < r[y]) then parent[x] = y;
12.     else{
13.       parent[y] = x;
14.       if (r[x] == r[y]) then r[x] = r[x] + 1;
15.     }
```

Disjoint set data structure

Disjoint Set: using 2 arrays:

- **parent[x]**: parent of node x on the tree containing x (if x is the root then parent[x] = x)
- **sz[x]**: number of nodes in the tree containing x

```

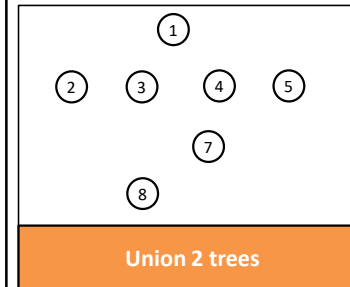
1. makeSet(x) { //Create a set containing x
2.     parent[x] = x; r[x] = 0;
3. }
4. find(x) { //return the identifier of the set containing x
5.     if (x != parent[x])
6.         parent[x] = find(parent[x]);
7.     return parent[x];
8. }
9. union(x, y) { //union 2 sets with identifiers x and y
10.    if (x != y)
11.        if (sz[x] < sz[y]) { sz[y] += sz[x]; parent[x] = y; }
12.        else {
13.            sz[x] += sz[y];
14.            parent[y] = x;
15.        }
16.    }
17. }

```

2. Longest consecutive subsequence

The problem: Given an unsorted array of pairwise distinct nonnegative integers, find the length of the longest consecutive elements sequence.

Example: nums = {1, 3, 5, 2, 7, 4, 8}

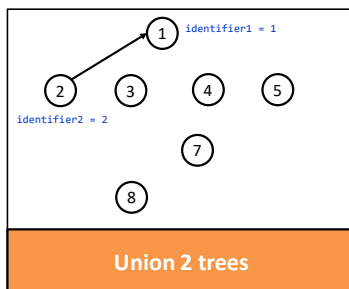


46

2. Longest consecutive subsequence

The problem: Given an unsorted array of pairwise distinct nonnegative integers, find the length of the longest consecutive elements sequence.

Example: nums = {1, 3, 5, 2, 7, 4, 8} k = 1 → Check 0 and 2



```

k = nums[i];
if (k - 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k-1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
if (k + 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k+1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)

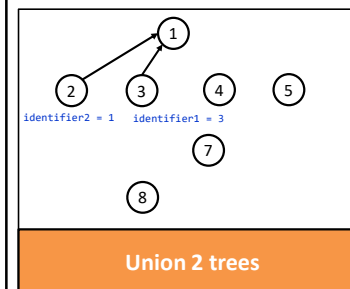
```

47

2. Longest consecutive subsequence

The problem: Given an unsorted array of pairwise distinct nonnegative integers, find the length of the longest consecutive elements sequence.

Example: nums = {1, 3, 5, 2, 7, 4, 8} k = 3 → Check 2 and 4



```

k = nums[i];
if (k - 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k-1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
if (k + 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k+1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)

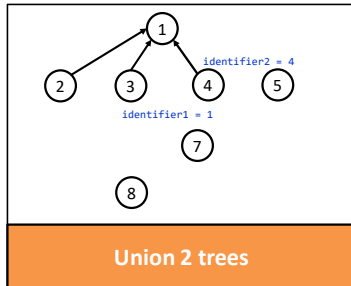
```

48

2. Longest consecutive subsequence

The problem: Given an unsorted array of pairwise distinct nonnegative integers, find the length of the longest consecutive elements sequence.

Example: nums = {1, 3, 5, 2, 7, 4, 8} $k = 3 \rightarrow$ Check 2 and 4



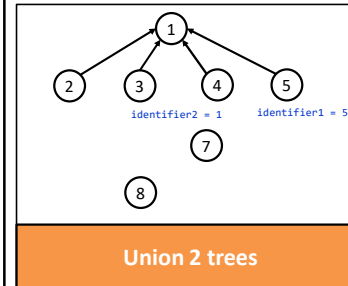
```
k = nums[i];
if (k - 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k-1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
if (k + 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k+1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
```

49

2. Longest consecutive subsequence

The problem: Given an unsorted array of pairwise distinct nonnegative integers, find the length of the longest consecutive elements sequence.

Example: nums = {1, 3, 5, 2, 7, 4, 8} $k = 5 \rightarrow$ Check 4 and 6



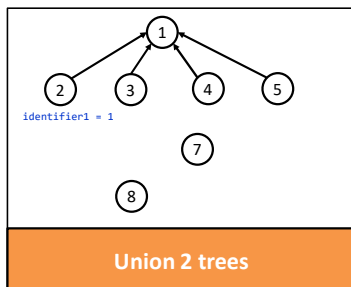
```
k = nums[i];
if (k - 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k-1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
if (k + 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k+1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
```

50

2. Longest consecutive subsequence

The problem: Given an unsorted array of pairwise distinct nonnegative integers, find the length of the longest consecutive elements sequence.

Example: nums = {1, 3, 5, 2, 7, 4, 8} $k = 2 \rightarrow$ Check 1 and 3



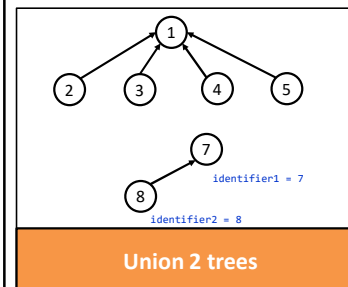
```
k = nums[i];
if (k - 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k-1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
if (k + 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k+1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
```

51

2. Longest consecutive subsequence

The problem: Given an unsorted array of pairwise distinct nonnegative integers, find the length of the longest consecutive elements sequence.

Example: nums = {1, 3, 5, 2, 7, 4, 8} $k = 7 \rightarrow$ Check 6 and 8



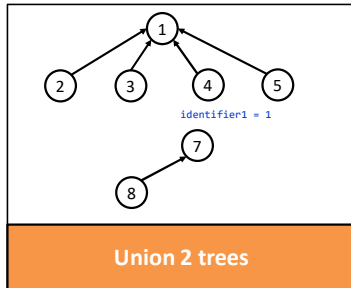
```
k = nums[i];
if (k - 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k-1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
if (k + 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k+1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
```

52

2. Longest consecutive subsequence

The problem: Given an unsorted array of pairwise distinct nonnegative integers, find the length of the longest consecutive elements sequence.

Example: nums = {1, 3, 5, 2, 7, 4, 8} k = 4 → Check 3 and 5



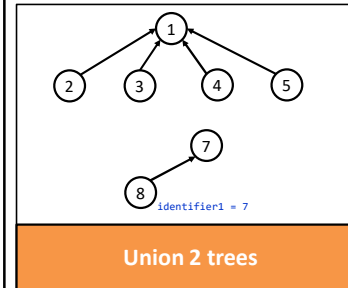
```
k = nums[i];
if (k - 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k-1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
if (k + 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k+1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
```

53

2. Longest consecutive subsequence

The problem: Given an unsorted array of pairwise distinct nonnegative integers, find the length of the longest consecutive elements sequence.

Example: nums = {1, 3, 5, 2, 7, 4, 8} k = 8 → Check 7 and 9



```
k = nums[i];
if (k - 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k-1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
if (k + 1) ∈ nums
    identifier1 = find(k);
    identifier2 = find(k+1);
    if (identifier1 != identifier2)
        union(identifier1, identifier2)
```

What is the answer ?
How to get the answer ?

54

Disjoint set data structure

Disjoint Set: using 2 arrays:

- parent[x]: parent of node x on the tree containing x (if x is the root then parent[x] = x)
- sz[x]: number of nodes in the tree containing x

```
1. makeSet(x) { //Create a set containing x
2.   parent[x] = x; sz[x] = 1;
3. }
4. find(x) { //return the identifier of the set containing x
5.   if (x != parent[x]) parent[x] = find(parent[x]);
6.   return parent[x];
7. }
8. union(x, y) { //union 2 sets with identifiers x and y
9.   if (sz[x] > sz[y]) { sz[x] += sz[y]; parent[y] = x; }
10.  else {
11.    sz[y] += sz[x];
12.    parent[x] = y;
13.  }
14. }
```

Disjoint set data structure

Disjoint Set: using 2 arrays:

- parent[x]: parent of node x on the tree containing x (if x is the root then parent[x] = x)
- sz[x]: number of nodes in the tree containing x

```
union(x, y) { //union 2 sets with identifiers x and y
    if (sz[x] > sz[y]) {
        sz[x] += sz[y];
        parent[y] = x;
        if (res < sz[x]) res = sz[x];
    }
    else {
        sz[y] += sz[x];
        parent[x] = y;
        if (res < sz[y]) res = sz[y];
    }
}
```

Variable **res**: store the result
Init: **res** = 0

```

void makeSet(int x) {
    parent[x] = x;
    sz[x] = 1;
}

void find(x) { //return the identifier of the set containing x
    if (x != parent[x]) parent[x] = find(parent[x]);
    return parent[x];
}

void union(x, y) { //union 2 sets with identifiers x and y
    if (sz[x] > sz[y]) {
        sz[x] += sz[y]; parent[y] = x;
        if (res < sz[x]) res = sz[x];
    }
    else {
        sz[y] += sz[x]; parent[x] = y;
        if (res < sz[y]) res = sz[y];
    }
}

void longestConsecutive(vector<int>& nums) {
    for (int i=0; i<nums.size(); i++) makeSet(nums[i]);
    for (int i=0; i<nums.size(); i++) {
        k = nums[i];
        if (k - 1) ∈ nums
            identifier1 = find(k); identifier2 = find(k-1);
            if (identifier1 != identifier2) union(identifier1, identifier2)
        if (k + 1) ∈ nums
            identifier1 = find(k); identifier2 = find(k+1);
            if (identifier1 != identifier2) union(identifier1, identifier2)
    }
}

```

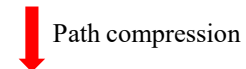
Summary: Disjoint set

- Path compression: to improve computation time of function find(x)

```

1. find(x){ //return id of the set containing x
2     while (x != parent[x]) do x = parent[x];
3.     return x;
4. }

```



```

find(x) {
    if (x != parent[x])
        parent[x] = find(parent[x]) #path compression
    return parent[x];
}

```

Summary: Disjoint set

1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2. if (x != y) then
3. parent[x] = y; //make id of x's set be the id of y's set



1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2. if (x != y) then
3. if (r[x] > r[y]) then parent[y] = x;
4. else{
5. parent[x] = y;
6. if (r[x] == r[y]) then r[y] = r[y] + 1; Union by rank to reduce the height of tree
7. }
8. }

Disjoint set data structure

CASE 1: If use union(x, y) not by rank/size and find(x) not compression:

- find(x): could take $O(|N|)$ in a worst-case scenario (a long, skinny tree).
- union(x, y): Each union(x, y) operation takes $O(1)$ because we just update the parent.

```

1. makeSet(x) { //Create a set containing x
2.     parent[x] = x; r[x] = 0;
3. }
4. find(x){ //return id of the set containing x; not compression
5     while (x != parent[x]) do x = parent[x];
6.     return x;
7. }
8. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y; not by rank
9.     if (x != y) then
10.         parent[x] = y; //make id of x's set be the id of y's set

```

Disjoint set data structure

CASE 2: If use union(x, y) by rank/size and find(x) not compression:

- find(x): $O(\log|N|)$
- union(x, y): $O(1)$ to update parent and rank.

```
1. makeSet(x) { //Create a set containing x
2.     parent[x] = x; r[x] = 0;
3. }
4. find(x) { //return id of the set containing x; not compression
5.     while (x != parent[x]) do x = parent[x];
6.     return x;
7. }
8. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y; union by rank
9.     if (x != y) then
10.         if (r[x] > r[y]) then parent[y] = x;
11.         else{
12.             parent[x] = y;
13.             if (r[x] == r[y]) then r[y] = r[y] + 1;
14.         }
15. }
```

Disjoint set data structure

CASE 3: If use union(x, y) by rank/size and find(x) path compression:

- find(x): $\alpha(|N|)$ where $\alpha(|N|)$ is the **inverse Ackermann function** (If $|N| \approx 10^{80}$ then $\alpha(|N|) \leq 5$), so find(x) takes almost constant $O(1)$.
- union(x, y): $O(1)$

```
1. makeSet(x) { //Create a set containing x
2.     parent[x] = x; r[x] = 0;
3. }
4. find(x) {
5.     if (x != parent[x])
6.         parent[x] = find(parent[x]) //path compression
7.     return parent[x];
8. }
9. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y; union by rank
10.     if (x != y) then
11.         if (r[x] > r[y]) then parent[y] = x;
12.         else{
13.             parent[x] = y;
14.             if (r[x] == r[y]) then r[y] = r[y] + 1;
15.         }
16. }
```

```
void makeSet(int x) {
    parent[x] = x;
    sz[x] = 1;
}
void find(x) { //return the identifier of the set containing x
    if (x != parent[x]) parent[x] = find(parent[x]);
    return parent[x];
}
void union(x, y) { //union 2 sets with identifiers x and y
    if (sz[x] > sz[y]) {
        sz[x] += sz[y]; parent[y] = x;
        if (res < sz[x]) res = sz[x];
    }
    else {
        sz[y] += sz[x]; parent[x] = y;
        if (res < sz[y]) res = sz[y];
    }
}
unordered_set<int> S(nums.begin(), nums.end());

void longestConsecutive(vector<int>& nums) {
    for (int i=0; i<nums.size(); i++) makeSet(nums[i]);
    for (int i=0; i<nums.size(); i++) {
        k = nums[i];
        if (k-1) ∈ nums → if (S.count(k-1)) 0(1)
            identifier1 = find(k); identifier2 = find(k-1);
            if (identifier1 != identifier2) union(identifier1, identifier2)
        if (k+1) ∈ nums → if (S.count(k+1)) 0(1)
            identifier1 = find(k); identifier2 = find(k+1);
            if (identifier1 != identifier2) union(identifier1, identifier2)
    }
}
```

Course outline

1. Duplicate Customer Account Consolidation
2. Longest consecutive subsequence
3. Minimum spanning tree - Kruskal algorithm
4. Islands
5. Friends

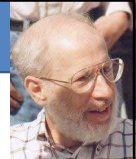
3. Minimum spanning tree - Kruskal algorithm

1. Duplicate Customer Account Consolidation
2. Longest consecutive subsequence
3. Minimum spanning tree - Kruskal algorithm
4. Islands
5. Friends

65

Kruskal algorithm

```
Generic-MST(G, c)
  T = ∅
  // T is the subset edges of a minimum spanning tree
  while T is not the spanning tree do
    Finding edge (u, v) is "safe" edge for T
    T = T ∪ {(u, v)}
  return T
```



Joseph Kruskal
(1928 - ~)

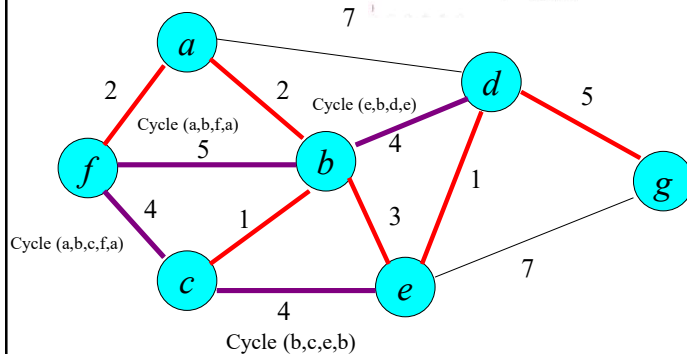
Kruskal algorithm:

- ❖ T is forest (empty).
- ❖ The "safe" edge included in T at each iteration is the edge with smallest weight among edges connecting its connected components.

```
void Kruskal ( ) {
  Sort |E| edges of the graph e1, e2, . . . , e|E| in ascending order of weight;
  T = ∅; // T: set of edges of the minimum spanning tree
  for (i = 1; i <= |E|; i++)
    if (T ∪ {ei} does not contain cycle)
      T = T ∪ {ei};
}
```

Kruskal algorithm: an example

```
void Kruskal ( ) {
  Sort |E| edges of the graph e1, e2, . . . , e|E| in ascending order of weight;
  T = ∅; // T: set of edges of the minimum spanning tree
  for (i = 1; i <= |E|; i++)
    if (T ∪ {ei} does not contain cycle)
      T = T ∪ {ei};
}
```



Weight of the minimum spanning tree:
2+2+1+3+1+5=14

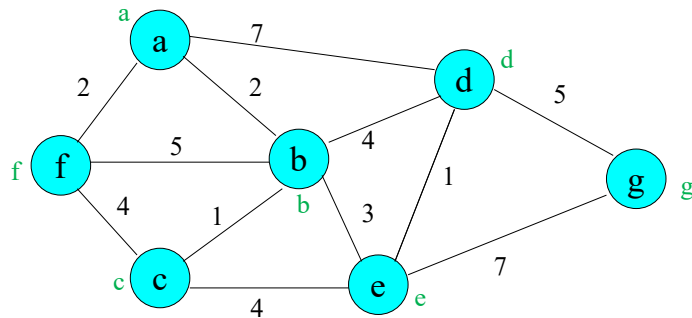
Computation time

```
void Kruskal ( )
{
  Sort |E| edges of the graph e1, e2, . . . , e|E| in ascending order of weight;
  T = ∅; // T: set of edges of the minimum spanning tree
  for (i = 1; i <= |E|; i++)
    if (T ∪ {ei} does not contain cycle)
      T = T ∪ {ei};
}
```

- Step 1. Sort the edges in ascending order of weight
 - Could use heap sort/merge sort : $O(|E| \log |E|)$
- Iterations: Each iteration we need to check whether $T \cup \{e_i\}$ contains the cycle?
 - Could use DFS to check with time $O(|V| + |E|)$.
 - Total time: $O(|E|(|E| + |V|))$

Computation time: $O(|E| \log |E| + |E|(|E| + |V|))$ where $|V|$, $|E|$ is the number of vertices and edges of the graph, respectively.

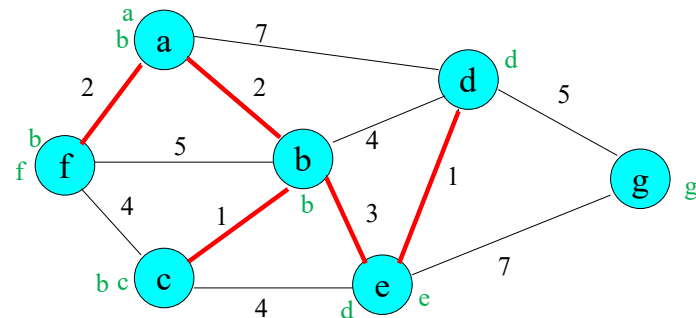
Kruskal – Disjoint set Example



69

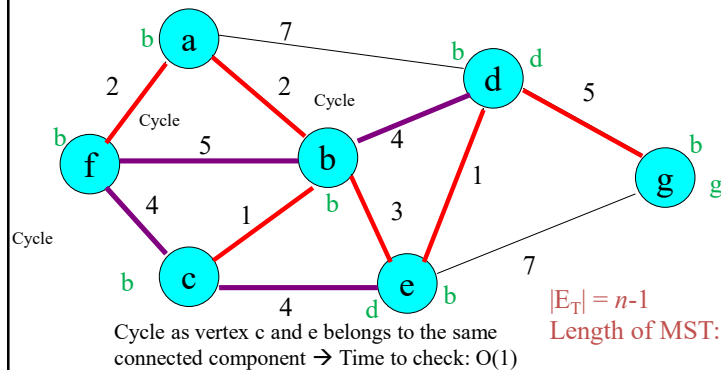
Kruskal – Disjoint set Example

Init: Each vertex initially forms its own connected component (a disjoint set).
The representative identifier of that component is the name of vertex



70

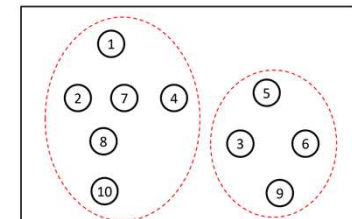
Kruskal – Disjoint set Example



71

Disjoint set

- Disjoint set data structure is used to represent sets that don't have any elements in common. It supports 3 main operations:
 - makeSet(x):** create a set containing only an element x
 - find (x):** find the id of a set containing element x
 - union(x, y):** merge the set having $id=x$ and the set having $id=y$ to a single set

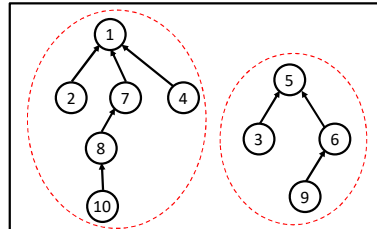


Two disjoint sets

Disjoint set

- Each set is represented by a tree:
 - Each node in the tree corresponds to an element of the set.
 - The root of the tree is the id of the set.
 - Each node has exactly one parent (parent of the root is itself).

x	1	2	3	4	5	6	7	8	9	10
parent[x]	1	1	5	1	5	5	1	7	6	8

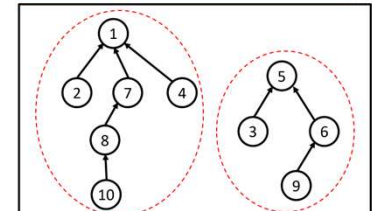


Each tree corresponds to a tree

Disjoint set

```

1. makeSet(x){ //Create a set containing only element x
2.     parent[x] = x;
3. }
4. find(x){ //return id of the set containing x
5.     while (x != parent[x]) do x = parent[x];
6.     return x;
7. }
    
```



Each tree corresponds to a tree

Disjoint set

```

1. makeSet(x){ //Create a set containing only element x
2.     parent[x] = x;
3. }
4. find(x){ //return id of the set containing x
5.     while (x != parent[x]) do x = parent[x];
6.     return x;
7. }
8. union(x, y) { //merge 2 sets: the set with id = x and the set with id = y
11.    if (x != y) then
12.        parent[x] = y; //make id of x's set be the id of y's set
    }
    
```

Kruskal: disjoint set

```

1. Kruskal(G=(V,E),w){
2.     E' = sort edges in ascending order of weight;
3.     Ef = ∅
4.     for (u, v) in E' do {
5.         if Ef ∪ (u, v) does not contain cycle then
6.             Ef = Ef ∪ (u, v);
7.             if (|Ef| == |V| - 1) then break;
8.         }
9.     }
10.    if (|Ef| < |V| - 1) then {
11.        print("ERROR: Graph G is not connected");
12.        return NULL;
13.    }
14.    else return (V, Ef);
15. }
    
```

Not use Disjoint set

$$O(|E| \log |E| + |E|(|E| + |V|))$$

```

1. Kruskal(G=(V,E),w){
2.     for (x = 1; x <= |V|; x++) do makeSet(x);
3.     E' = sort edges in ascending order of weight;
4.     Ef = ∅
5.     for (u, v) in E' do {
6.         idu = find(u);
7.         idv = find(v);
8.         if (idu != idv) then
9.             union(idu, idv);
10.            Ef = Ef ∪ (u, v);
11.            if (|Ef| == |V| - 1) then break;
12.        }
13.    }
14.    if (|Ef| < |V| - 1) then {
15.        print("ERROR: Graph G is not connected");
16.        return NULL;
17.    }
18.    else return (V, Ef);
19. }
    
```

Use Disjoint set

What is the computation time if disjoint set is used? $O(|V| + |E| \log |E| + |E||V|)$

Kruskal: disjoint set

```
makeSet(x){ //Create a set containing only element x
    parent[x] = x;
}
find(x){ //not compression
    while (x != parent[x]) do x = parent[x];
    return x;
}
union(x, y) { //not union by rank
    if (x != y) then
        parent[x] = y;
}
```

not optimize

$O(|V| + |E|\log|E| + |E||V|)$

```
1. Kruskal(G=(V,E),w){
2.   for (x = 1; x <= |V|; x++) do makeSet(x);
3.   E' = sort edges in ascending order of weight;
4.   Ei = ∅
5.   for (u, v) in E' do {
6.       idu = find(u);
7.       idv = find(v);
8.       if (idu != idv) then
9.           union(idu, idv);
10.      Ei = Ei ∪ (u,v);
11.      if (|Ei| == |V| - 1) then break;
12.   }
13. }
14. if (|Ei| < |V| - 1) then {
15.   print("ERROR: Graph G is not connected");
16.   return NULL;
17. }
18. else return (V, Ei);
19. }
```

Line 2: $O(|V|)$
 Line 3: $O(|E|\log|E|)$
 Line 6-7: $O(|V|)$
 Line 9: $O(1)$

If use union(x, y) not by rank and find(x) not compression:

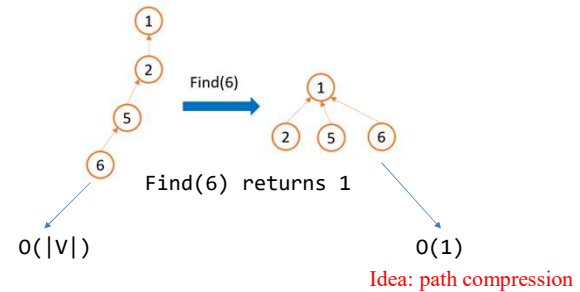
- **find(x)**: could take $O(|V|)$ in a worst-case scenario (a long, skinny tree).
- **union(x, y)**: Each union(x, y) operation takes $O(1)$ because we just update the parent.

→ Kruskal: We call find(x) multiple times → the loop lines 5-13: the overall cost can be $O(|E||V|)$.

77

Kruskal: disjoint set

How to improve the complexity of function **find(x)** ?

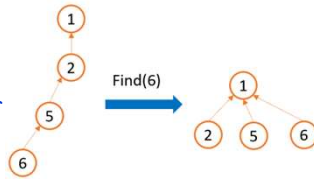


78

Disjoint set data structure

find(x): returns the identifier of the set containing x

- Starting from x, continuously access the parent node to find the root node
- Complexity is proportional to the length of the path from x to the origin
- **Path Compression**: in the process of going from x to the root, nodes on the path will be attached as direct children of the root node → reduce the path length from these nodes to the root in the following iterations.



```
find(x){
    while (x != parent[x])
        x = parent[x];
    return x;
}

find(x){
    if(x != parent[x])
        parent[x] = find(parent[x]);
    return parent[x];
}
```

Disjoint set

- Path compression: to improve computation time of function **find(x)**

```
1. find(x){ //return id of the set containing x
2.   while (x != parent[x]) do x = parent[x];
3.   return x;
4. }
```

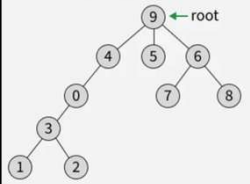
Path compression

```
find(x) {
    if (x != parent[x])
        parent[x] = find(parent[x]) //path compression
    return parent[x];
}
```

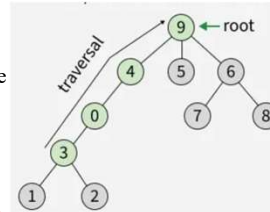
Disjoint set: Path compression

```
find(x) {
  if (x != parent[x])
    parent[x] = find(parent[x])  #path compression
  return parent[x];
}
```

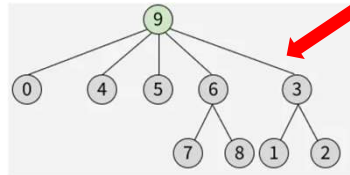
Set {0, 1, ..., 9} is represented as



Call find(3): traverse up and find 9 is id of the tree



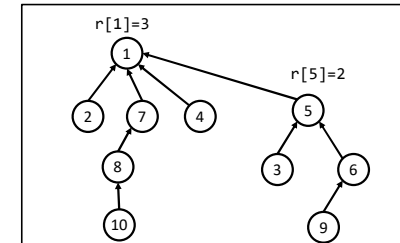
- The idea of path compression is to make the found root as parent of x so that we don't have to traverse all intermediate nodes again. If x is root of a subtree, then path (to root) from all nodes under x also compresses. → It speeds up the data structure by compressing the height of the trees.



When find(3) is terminated, path compression is already applied → we get directly link element 0 and 3 to 9

Disjoint set

- Reduce the size of each tree: to improve computation time of function union(x, y)
 - When merging two trees, we will join the tree with the lower height to the tree with the higher height.
 - We use the additional variable r[x] to record the height of a node x in the tree



Union 2 disjoint sets

Disjoint set

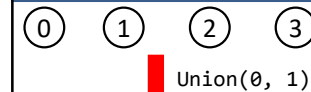
- union(x, y) //merging 2 sets: the set with id=x and the set with id=y
- if (x != y) then
- parent[x] = y; //make id of x's set be the id of y's set



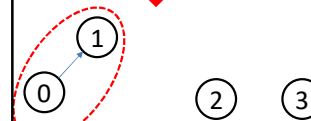
- union(x, y) //merging 2 sets: the set with id=x and the set with id=y
- if (x != y) then
- if (r[x] > r[y]) then parent[y] = x;
- else{
- parent[x] = y;
- if (r[x] == r[y]) then r[y] = r[y] + 1; Union by rank to reduce the height of tree
- }
- }

Disjoint set

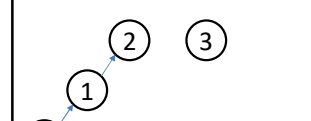
- union(x, y) //merging 2 sets: the set with id=x and the set with id=y
- if (x != y) then
- parent[x] = y; //make id of x's set be the id of y's set



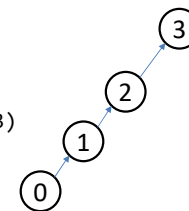
Union(0, 1)



Union(1, 2)



Union(2, 3)



Disjoint set

```

1. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y
2.   if (x != y) then
3.     if (r[x] > r[y]) then parent[y] = x;
4.   else{
5.     parent[x] = y;
6.     if (r[x] == r[y]) then r[y] = r[y] + 1; Union by rank to reduce
7.   }                                     the height of tree
8. }

```

Union(0, 1)

Union(1, 2)

Union(1, 3)

Kruskal: disjoint set

```

1. Kruskal(G=(V,E),w){
2.   E' = sort edges in ascending order of weight;
3.   Er = ∅
4.   for (u, v) in E' do {
5.     if Er ∪ (u, v) does not contain cycle) then
6.       Er = Er ∪ (u,v);
7.     if (|Er| == |V| - 1) then break;
8.   }
9. }
10. if (|Er| < |V| - 1) then {
11.   print("ERROR: Graph G is not connected");
12.   return NULL;
13. }
14. else return (V, Er);
15. }

```

86

Kruskal: disjoint set

```

1. Kruskal(G=(V,E),w){
2.   E' = sort edges in ascending order of weight;
3.   Er = ∅
4.   for (u, v) in E' do {
5.     if Er ∪ (u, v) does not contain cycle) then
6.       Er = Er ∪ (u,v);
7.     if (|Er| == |V| - 1) then break;
8.   }
9. }
10. if (|Er| < |V| - 1) then {
11.   print("ERROR: Graph G is not connected");
12.   return NULL;
13. }
14. else return (V, Er);
15. }

```

Not use Disjoint set

$O(|E| \log|E| + |E|(|E| + |V|))$

```

1. Kruskal(G=(V,E),w){
2.   for (x = 1; x <= |V|; x++) do makeSet(x);
3.   E' = sort edges in ascending order of weight;
4.   Er = ∅
5.   for (u, v) in E' do {
6.     idu = find(u);
7.     idv = find(v);
8.     if (idu != idv) then {
9.       union(idu, idv);
10.      Er = Er ∪ (u,v);
11.      if (|Er| == |V| - 1) then break;
12.    }
13.  }
14.  if (|Er| < |V| - 1) then {
15.    print("ERROR: Graph G is not connected");
16.    return NULL;
17.  }
18.  else return (V, Er); Use Disjoint set
19. }

```

What is the computation time if disjoint set is used ?

$O(|V| + |E| \log|E| + |E||V|)$

$O(|V| + |E| \log|E| + |E| \log|V|)$

$O(|V| + |E| \log|E| + |E| \alpha(|V|))$

87

Kruskal: disjoint set

```

makeSet(x) { //Create a set containing only element x
  parent[x] = x;
}

find(x) { //not compression
  while (x != parent[x]) do x = parent[x];
  return x;
}

union(x, y) { //not union by rank
  if (x != y) then
    parent[x] = y;
}

```

not optimize

$O(|V| + |E| \log|E| + |E||V|)$

```

1. Kruskal(G=(V,E),w){
2.   for (x = 1; x <= |V|; x++) do makeSet(x);
3.   E' = sort edges in ascending order of weight;
4.   Er = ∅
5.   for (u, v) in E' do {
6.     idu = find(u);
7.     idv = find(v);
8.     if (idu != idv) then {
9.       union(idu, idv);
10.      Er = Er ∪ (u,v);
11.      if (|Er| == |V| - 1) then break;
12.    }
13.  }
14.  if (|Er| < |V| - 1) then {
15.    print("ERROR: Graph G is not connected");
16.    return NULL;
17.  }
18.  else return (V, Er);
19. }

```

If use union(x, y) not by rank and find(x) not compression:

- **find(x):** could take $O(|V|)$ in a worst-case scenario (a long, skinny tree).
- **union(x, y):** Each union(x, y) operation takes $O(1)$ because we just update the parent.

→ Kruskal: We call find(x) multiple times → the loop lines 5-13: the overall cost can be $O(|E||V|)$.

88

Kruskal: disjoint set

```

makeSet(x) { //Create a set containing only element x
    parent[x] = x; r[x] = 0;
}

find(x) { //not compression
    while (x != parent[x]) do x = parent[x];
    return x;
}

union(x, y) { //union by rank
    if (x != y) then
        if (r[x] > r[y]) then parent[y] = x;
        else{
            parent[x] = y;
            if (r[x] == r[y]) then r[y] = r[y] + 1;
        }
    }
}

```

```

1. Kruskal(G=(V,E),w){
2.   for (x = 1; x <= |V|; x++) do makeSet(x);
3.   E' = sort edges in ascending order of weight;
4.   Ei = ∅
5.   for (u, v) in E' do {
6.     idu = find(u);
7.     idv = find(v);
8.     if (idu != idv) then {
9.       union (idu, idv);
10.      Ei = Ei ∪ {u,v};
11.      if (|Ei| == |V| - 1) then break;
12.    }
13.  }
14.  if (|Ei| < |V| - 1) then {
15.    print("ERROR: Graph G is not connected");
16.    return NULL;
17.  }
18.  else return (V, Ei);
19. }

```

$$O(|V| + |E|\log|E| + |E|\log|V|)$$

If use union(x, y) by rank and find(x) not compression:

- find(x): $O(\log|V|)$
- union(x, y): $O(1)$ to update parent and rank.

89

Kruskal: disjoint set

```

makeSet(x) { //Create a set containing only element x
    parent[x] = x; r[x] = 0;
}

find(x) { //path compression
    if (x != parent[x])
        parent[x] = find(parent[x])
    return parent[x];
}

union(x, y) { //union by rank
    if (x != y) then
        if (r[x] > r[y]) then parent[y] = x;
        else{
            parent[x] = y;
            if (r[x] == r[y]) then r[y] = r[y] + 1;
        }
    }
}

```

```

1. Kruskal(G=(V,E),w){
2.   for (x = 1; x <= |V|; x++) do makeSet(x);
3.   E' = sort edges in ascending order of weight;
4.   Ei = ∅
5.   for (u, v) in E' do {
6.     idu = find(u);
7.     idv = find(v);
8.     if (idu != idv) then {
9.       union (idu, idv);
10.      Ei = Ei ∪ {u,v};
11.      if (|Ei| == |V| - 1) then break;
12.    }
13.  }
14.  if (|Ei| < |V| - 1) then {
15.    print("ERROR: Graph G is not connected");
16.    return NULL;
17.  }
18.  else return (V, Ei);
19. }

```

$$O(|V| + |E|\log|E| + |E|\alpha(|V|))$$

If use union(x, y) by rank and find(x) path compression:

- find(x): $\alpha(|V|)$ where $\alpha(|V|)$ is the **inverse Ackermann function** (If $|V| \approx 10^{80}$ then $\alpha(|V|) \leq 5$), so find(x) takes almost constant $O(1)$.
- union(x, y): $O(1)$

90

Summary: Disjoint set data structure

CASE 1: If use union(x, y) not by rank/size and find(x) not compression:

- find(x): could take $O(|N|)$ in a worst-case scenario (a long, skinny tree).
- union(x, y): Each union(x, y) operation takes $O(1)$ because we just update the parent.

```

1. makeSet(x) { //Create a set containing x
2.   parent[x] = x; r[x] = 0;
3. }
4. find(x) { //return id of the set containing x; not compression
5.   while (x != parent[x]) do x = parent[x];
6.   return x;
7. }
8. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y; not by rank
9.   if (x != y) then
10.     parent[x] = y; //make id of x's set be the id of y's set

```

Summary: Disjoint set data structure

CASE 2: If use union(x, y) by rank/size and find(x) not compression:

- find(x): $O(\log|N|)$
- union(x, y): $O(1)$ to update parent and rank.

```

1. makeSet(x) { //Create a set containing x
2.   parent[x] = x; r[x] = 0;
3. }
4. find(x) { //return id of the set containing x; not compression
5.   while (x != parent[x]) do x = parent[x];
6.   return x;
7. }
8. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y; union by rank
9.   if (x != y) then
10.     if (r[x] > r[y]) then parent[y] = x;
11.     else{
12.       parent[x] = y;
13.       if (r[x] == r[y]) then r[y] = r[y] + 1;
14.     }
15. }

```

Summary: Disjoint set data structure

CASE 3: If use union(x, y) by rank/size and find(x) path compression:

- find(x): $\alpha(|N|)$ where $\alpha(|N|)$ is the **inverse Ackermann function** (If $|N| \approx 10^{80}$ then $\alpha(|N|) \leq 5$), so find(x) takes almost constant $O(1)$.
- union(x, y): $O(1)$

```
1. makeSet(x) { //Create a set containing x
2.     parent[x] = x; r[x] = 0;
3. }
4. find(x) {
5.     if (x != parent[x])
6.         parent[x] = find(parent[x])    //path compression
6.     return parent[x];
8. }
9. union(x, y) { //merging 2 sets: the set with id=x and the set with id=y; union by rank
10.    if (x != y) then
11.        if (r[x] > r[y]) then parent[y] = x;
12.        else{
13.            parent[x] = y;
14.            if (r[x] == r[y]) then r[y] = r[y] + 1;
15.        }
16. }
```